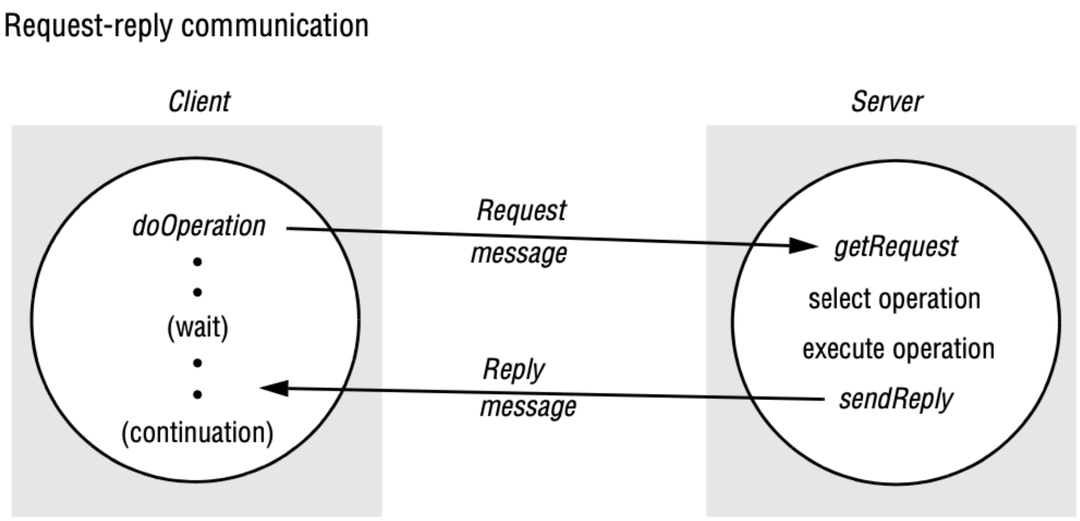
Comunicación Distribuida entre Procesos

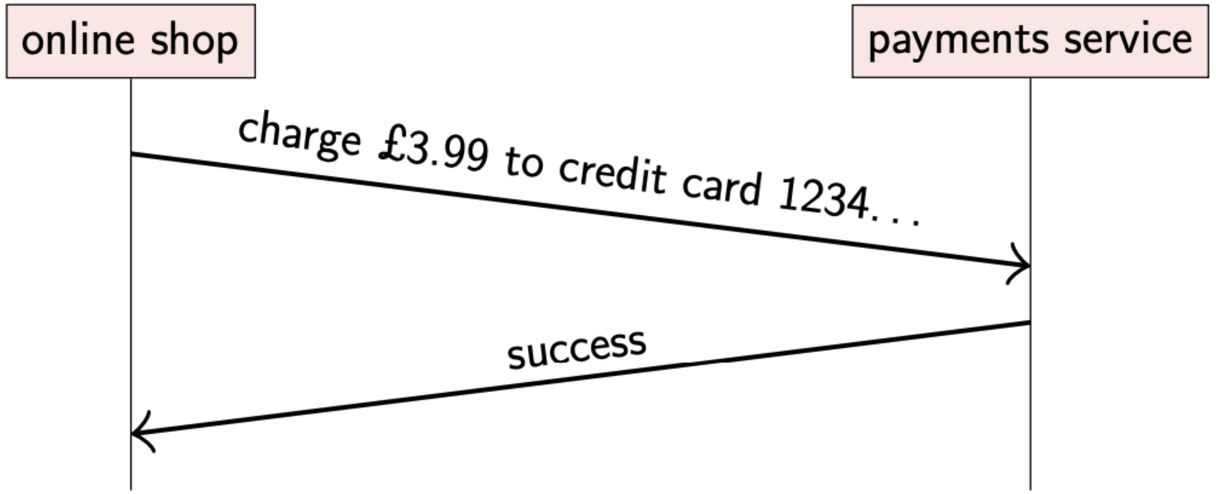
Request - Reply Communication

- Representa un patrón de comunicación en el contexto de las aplicaciones distribuidas.
- Normalmente se emplea en el contexto de aplicaciones cliente/servidor.
- Normalmente la comunicación es de naturaleza sincrónica.
- Se puede considerar confiable debido a que la respuesta tiende a ser un acknowledge.
- Ejemplo: HTTP



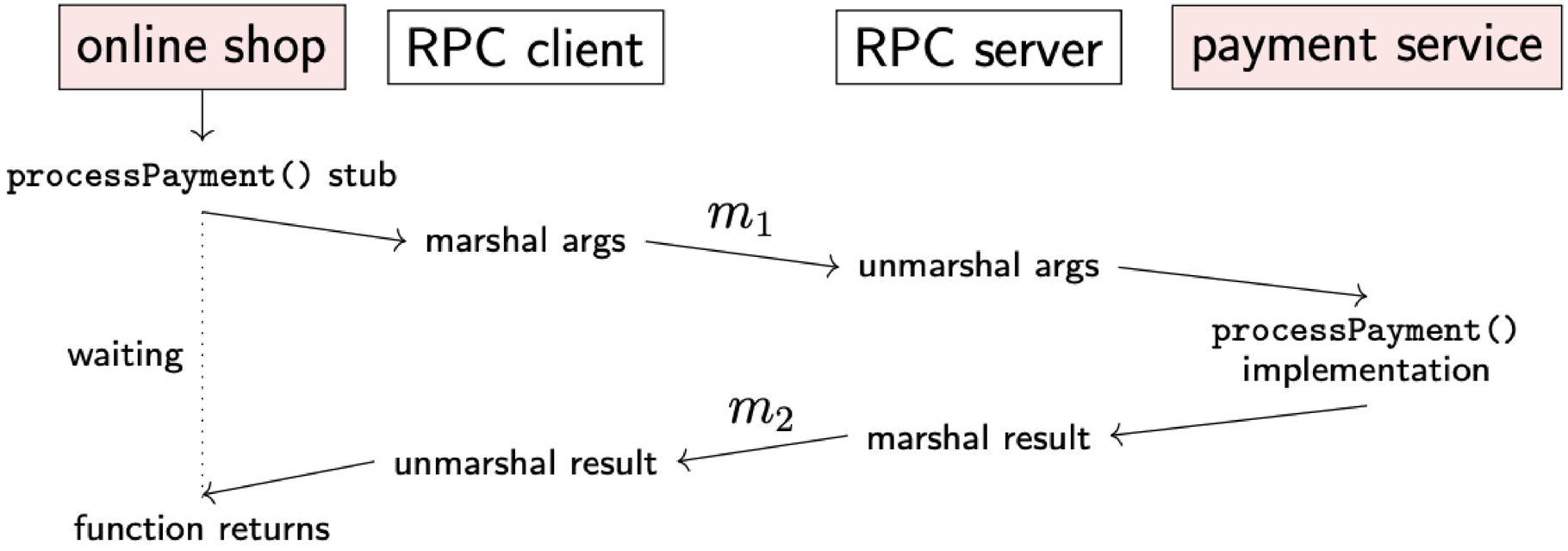
Example: Remote Procedure Call

Client-server example: online payments



Remote Procedure Call (RPC)

- Las Llamadas a Procedimiento Remoto (RPC) es un protocolo de comunicación que permite que un programa solicite un servicio de un programa en otra computadora a través de una red.
  - Idea principal: Hacer la comunicación entre procesos distribuidos de tal manera como se hace la llamada a un procedimiento normal.
  - De esta forma RPCs implica invocar procedimientos o funciones en una máquina remota como si fueran locales, abstrayendo las complejidades de la comunicación en red.



$m_1 =$

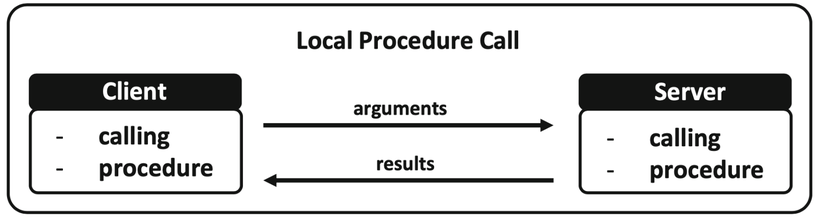
```
{
  "request": "processPayment",
  "card": {
    "number": "1234567887654321",
    "expiryDate": "10/2024",
    "CVC": "123"
  },
  "amount": 3.99,
  "currency": "GBP"
}
```

$m_2 =$

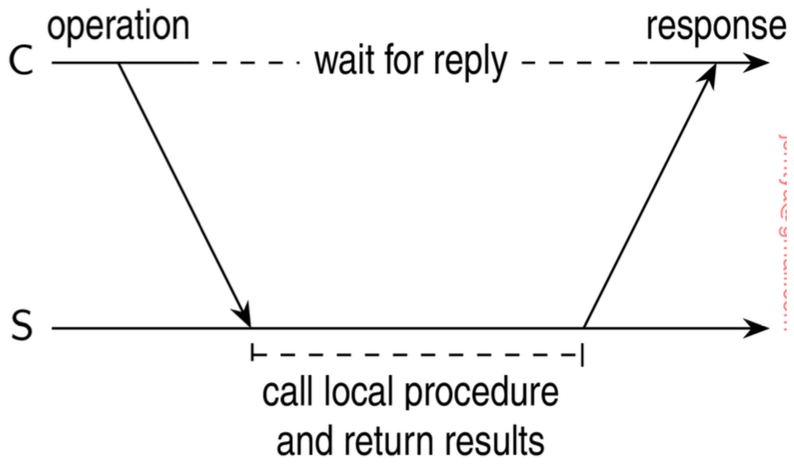
```
{
  "result": "success",
  "id": "XP61hHw2Rvo"
}
```

¿Cómo Funciona RPCs?

- Arquitectura Cliente-Servidor: RPC opera en un modelo cliente-servidor, donde el cliente llama a procedimientos ubicados en el servidor.
- Invocación de Procedimiento por el Cliente: El cliente invoca un procedimiento en el servidor como si fuera una llamada local.
- Marshalling: Los datos se empaquetan para su transmisión desde el cliente al servidor.
- Transmisión sobre la Red: Los datos empaquetados se envían por la red.
- Unmarshalling: El servidor desempaqueta los datos y ejecuta el procedimiento solicitado.



Llamada a Procedimientos Locales



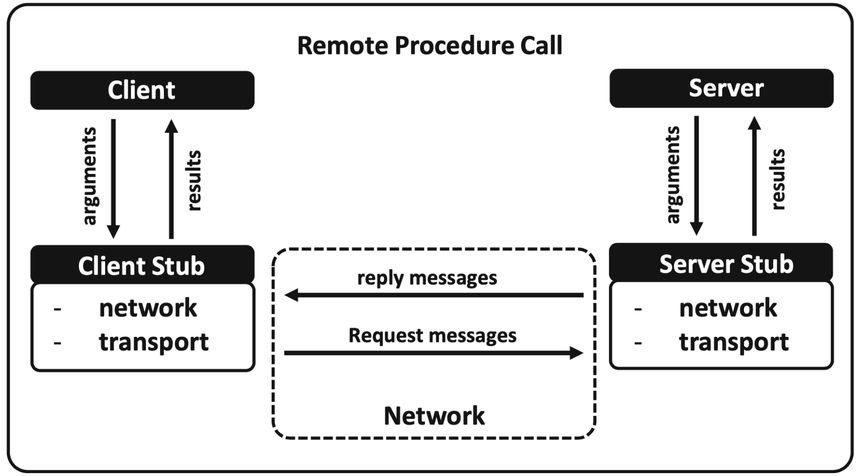
Remote Procedure Call Example

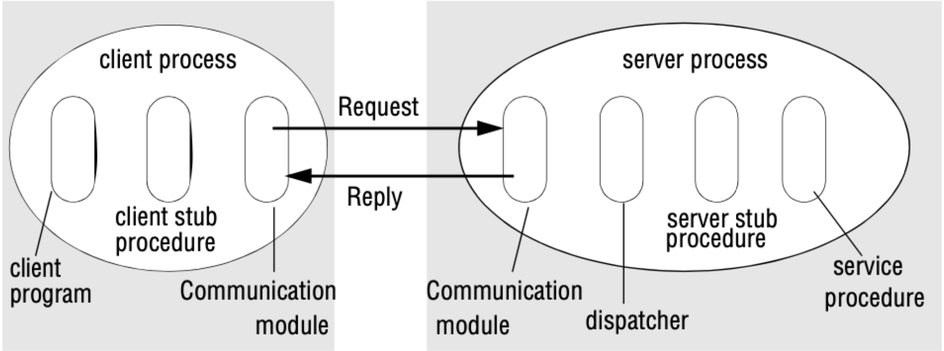
```
// Online shop handling customer's card details
Card card = new Card();
card.setCardNumber("1234 5678 8765 4321");
card.setExpiryDate("10/2024");
card.setCVC("123");

Result result = paymentsService.processPayment(card,
    3.99, Currency.GBP);

if (result.isSuccess()) {
    fulfilOrder();
}
```

Implementation of this function is on another node!

Elementos de RPCs

- Cliente Stub: Actúa como una interfaz entre el cliente y el servidor, permitiendo la invocación local de procedimientos remotos.
- Server Stub o Skeleton: Facilita la comunicación en el lado del servidor, manejando las solicitudes entrantes y dirigiéndolas al procedimiento real.
- Marshaller/Unmarshaller: Es responsable de serializar y deserializar datos para una transmisión eficiente.
- Protocolo RPC: Define las reglas y convenciones para la comunicación entre el cliente y el servidor.





Representación Externa de Datos - Marshalling

- En el proceso de intercambio de datos, una de los siguientes métodos puede ser utilizado para intercambiar datos en binarios:
  - Los valores se convierten a un formato externo acordado antes de la transmisión de los datos y se convierten al formato local al recibirlos.
  - Los valores pueden ser transmitidos en el formato del transmisor, junto con un flag del formato que esta siendo utilizado.

Marshalling/Unmarshalling

- Es el proceso de tomar una colección de elementos de datos y ensamblarlos en una forma adecuada para la transmisión en un mensaje.
- Unmarshalling es el proceso de desensamblarlos a la llegada para producir una colección equivalente de elementos de datos en el destino

Tipos de RPCs

- RPC Síncrono: Llamada bloqueante donde el cliente espera la respuesta del servidor.
- RPC Asíncrono: Llamada no bloqueante donde el cliente continúa la ejecución sin esperar la respuesta del servidor.
- RPC Unidireccional: El cliente envía una solicitud al servidor sin esperar una respuesta.
- RPC Bidireccional: Comunicación de dos vías, donde tanto el cliente como el servidor pueden iniciar solicitudes.

RPC: Seis Pasos

1. El cliente llama cliente stub. La llamada es una llamada a un procedimiento local.

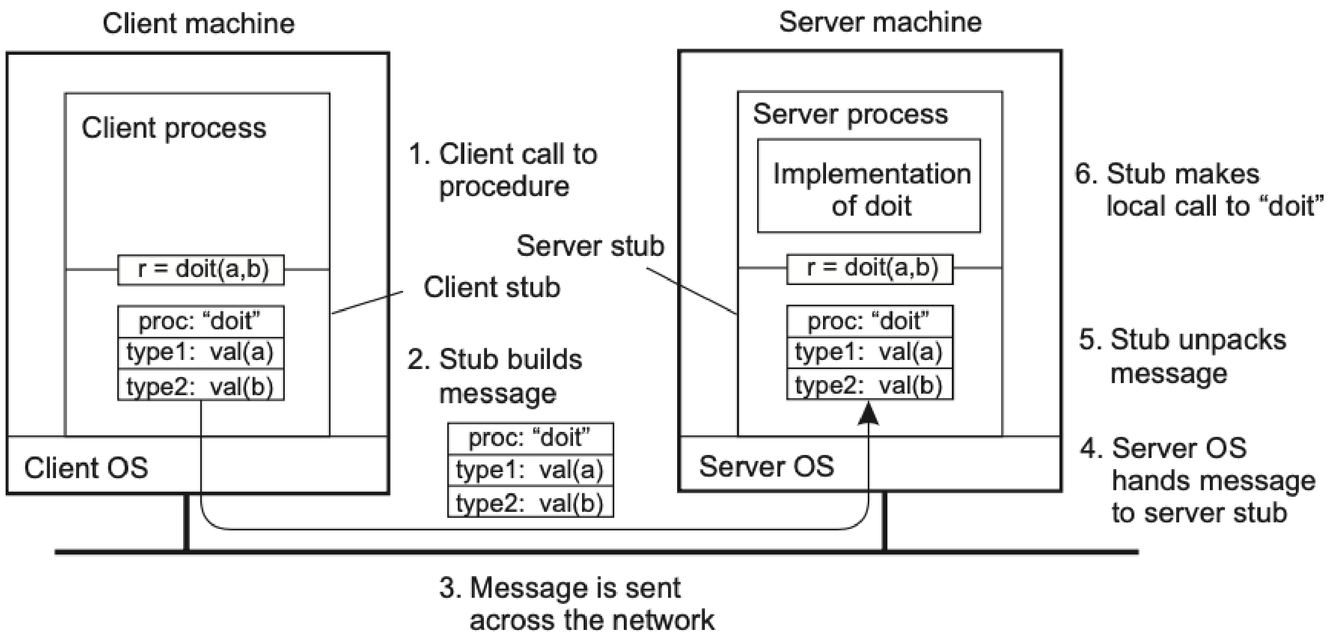
2. El cliente stub empaqueta los parámetros en un mensaje y realiza una llamada al sistema para enviar el mensaje. Empaquetar los parámetros se llama marshalling.

3. El sistema operativo local del cliente envía el mensaje desde la máquina cliente a la máquina servidor.

4. El sistema operativo local en la máquina del servidor pasa los paquetes entrantes al código stub del servidor.

5.El código stub del servidor descomprime los parámetros del mensaje. Desempaqueta los parámetros.

6.Finalmente, el código stub del servidor llama al procedimiento del servidor. La respuesta sigue los mismos pasos en sentido inverso.

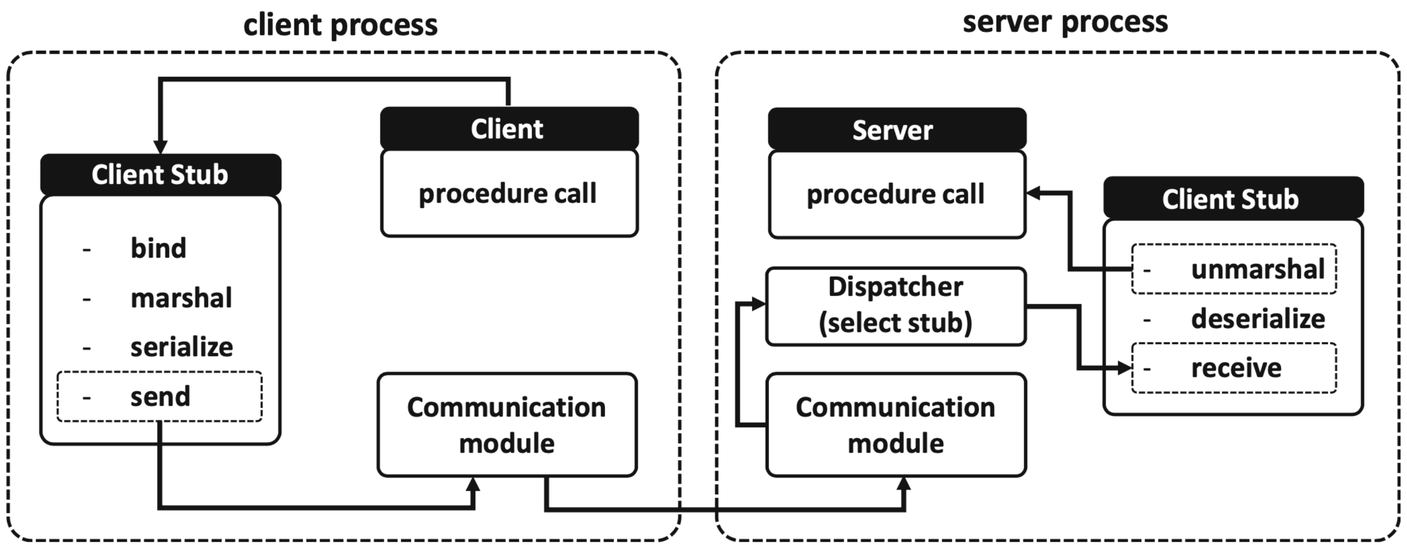


Stubs

- Un Stub en RPC actúa como intermediario entre el cliente y el procedimiento remoto real en el servidor.
- El stub permite al cliente invocar procedimientos remotos como si fueran locales, abstrayendo las complejidades de la comunicación en red.
- Rol del Stub:
  - Facilitación de la Interfaz: Sirve como un proxy que imita la interfaz del procedimiento remoto.
  - Empaquetado y Desempaquetado: Es responsable de empaquetar y desempaquetar datos para la transmisión a través de la red.
  - Invocación de Método: Traduce las llamadas de procedimientos locales en llamadas de procedimientos remotos.

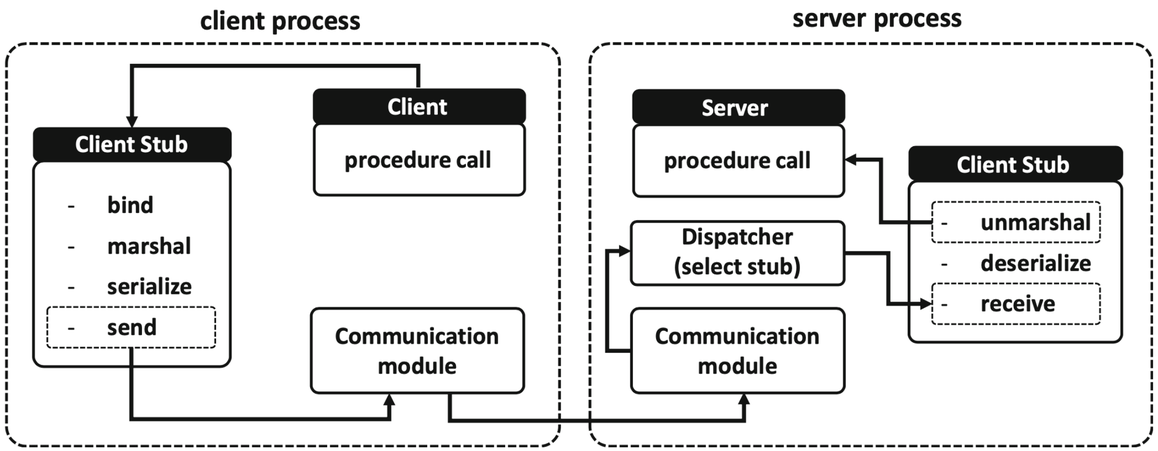
Tipos de Stub

- Stub del Cliente: Reside en el lado del cliente, intercepta las llamadas de procedimientos locales y las prepara para su transmisión al servidor.
- Stub del Servidor: Reside en el lado del servidor, recibe las solicitudes entrantes, desempaqueta los datos y los dirige al procedimiento remoto real.



Stubs: Flujo de Trabajo

- Invocación del Cliente:
  - El cliente invoca un procedimiento remoto como si fuera local.
  - El Stub intercepta la llamada y prepara los datos para su transmisión.
- Empaquetado (Marshalling):
  - Los parámetros se empaquetan en un formato adecuado para la transmisión por la red.
  - Los datos se preparan para una comunicación segura y eficiente.
- Transmisión:
  - Los datos empaquetados se transmiten por la red al servidor.
- Envío de los datos:
  - Se invoca el módulo de comunicación para la transmisión del mensaje serializado.
- Desempaquetado (Unmarshalling):
  - El Stub del Servidor recibe los datos y los desempaqueta.
  - Los parámetros se extraen para su uso por el procedimiento remoto real.
- Ejecución del Procedimiento Remoto:
  - Los datos desempaquetados se pasan al procedimiento remoto real para su ejecución.
  - Los resultados, si los hay, se empaquetan y envían de vuelta al cliente.

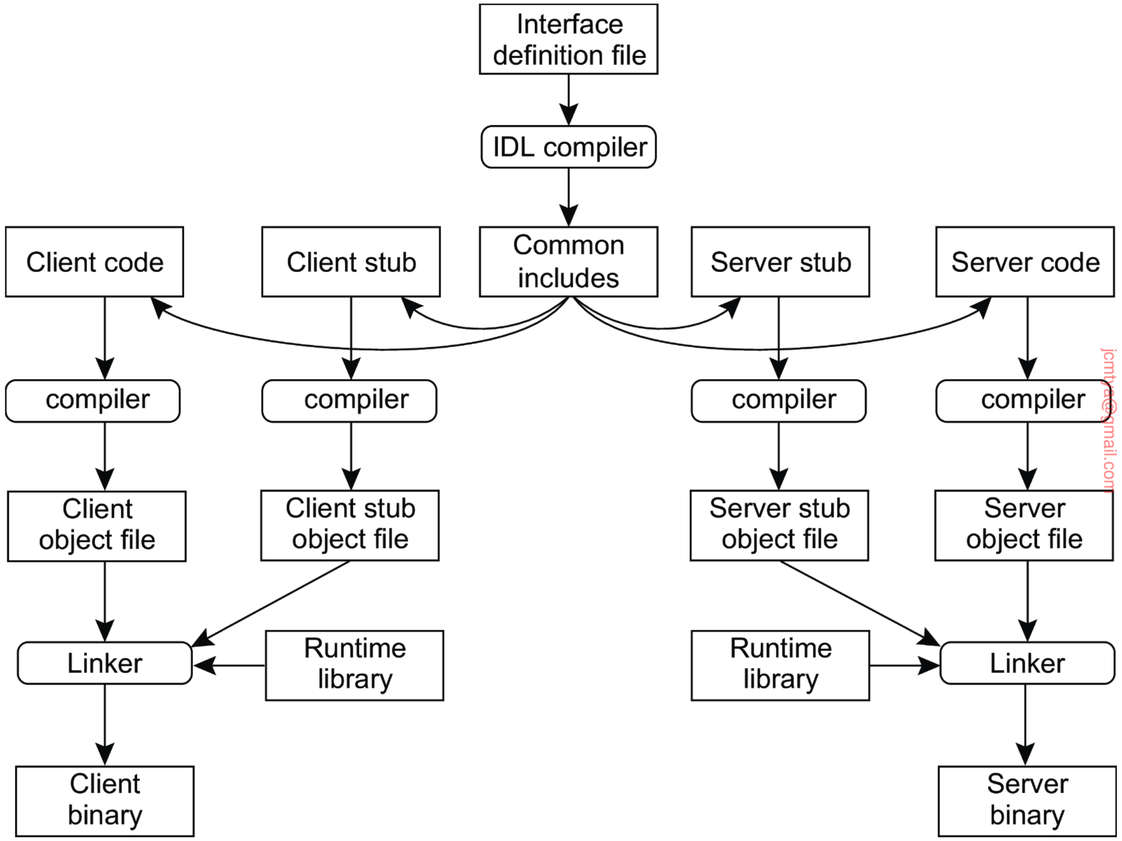


¿Por qué usar Stubs?

- Abstracción: Los clientes interactúan con procedimientos remotos como si fueran locales, abstrayendo las complejidades de la red.
- Compatibilidad: Los Stubs aseguran que los formatos de datos sean compatibles entre diferentes sistemas.
- Transparencia: Los clientes y servidores pueden implementarse en diferentes lenguajes o en plataformas distintas.

¿Cómo se genera el Stub?

- El primer paso es definir la interfaz utilizando un Lenguaje de Definición de Interfaz (IDL).



Lenguajes de Definición de Interfaz

- Los Lenguajes de Definición de Interfaz (IDL) en RPC son herramientas que permiten describir la interfaz entre clientes y servidores en un entorno distribuido.
- Su objetivo es facilitar la comunicación e interoperabilidad entre sistemas heterogéneos.
- La importancia de éstos radica en que establecen un contrato claro entre las partes, especificando los métodos, tipos de datos y protocolos de comunicación.

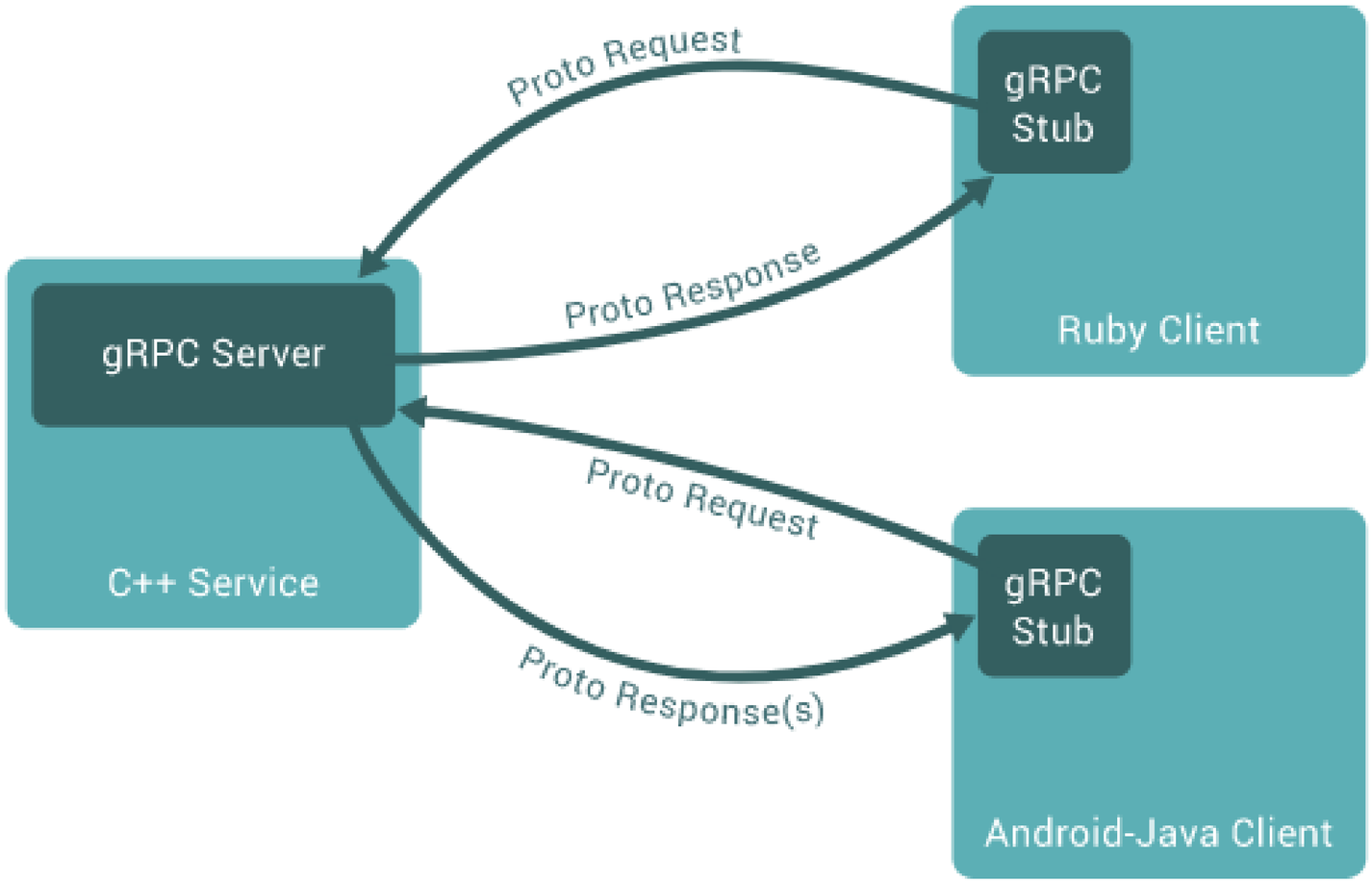
IDL en RPCs

- Interfaz Uniforme: Define cómo los componentes de software se comunicarán entre sí.
- Independencia de Plataforma: Permite que clientes y servidores escritos en diferentes lenguajes y ejecutándose en diferentes plataformas se comuniquen sin problemas.
- Generación Automática de Código: Las herramientas de IDL pueden generar automáticamente el código necesario para la comunicación cliente-servidor.

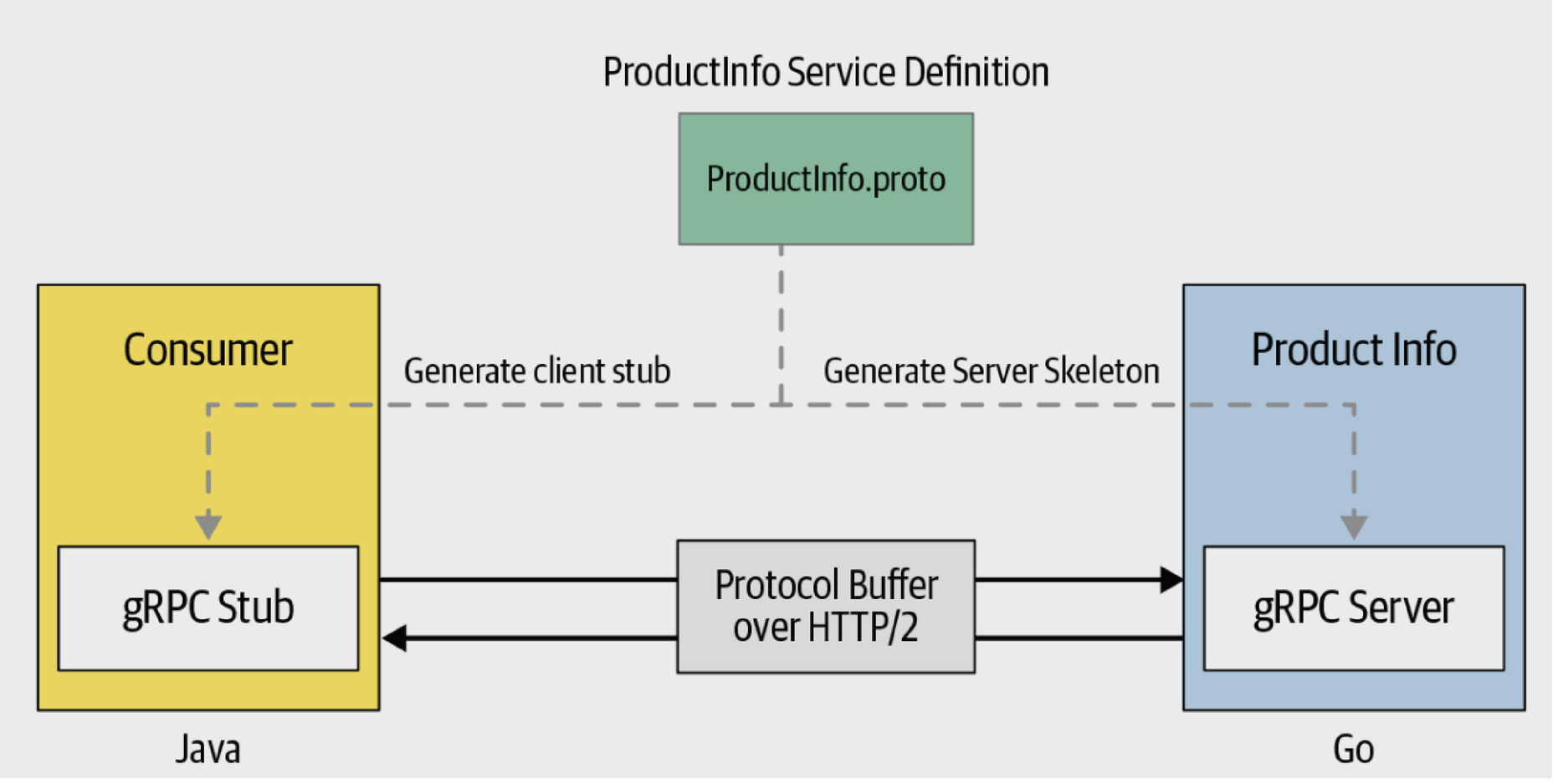
Tipos y Ejemplos de IDL

- IDL Tradicional: Ejemplos incluyen CORBA IDL y DCOM IDL.
- Protocol Buffers: Utilizado en gRPC, un sistema de RPC desarrollado por Google.
- Swagger/OpenAPI: Utilizado para describir y documentar interfaces de servicios web RESTful.
- GraphQL SDL: Utilizado en GraphQL para definir esquemas de datos.

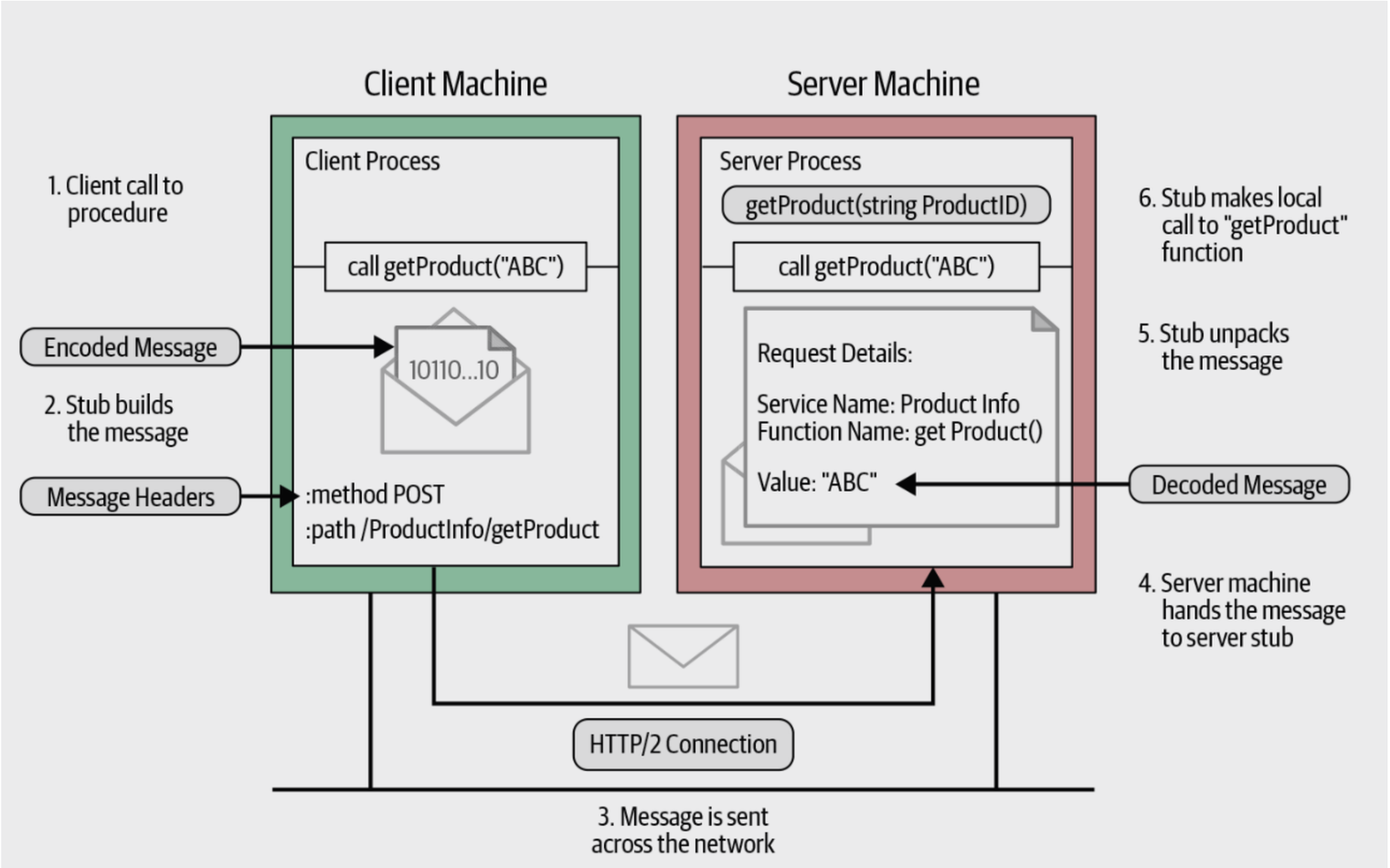
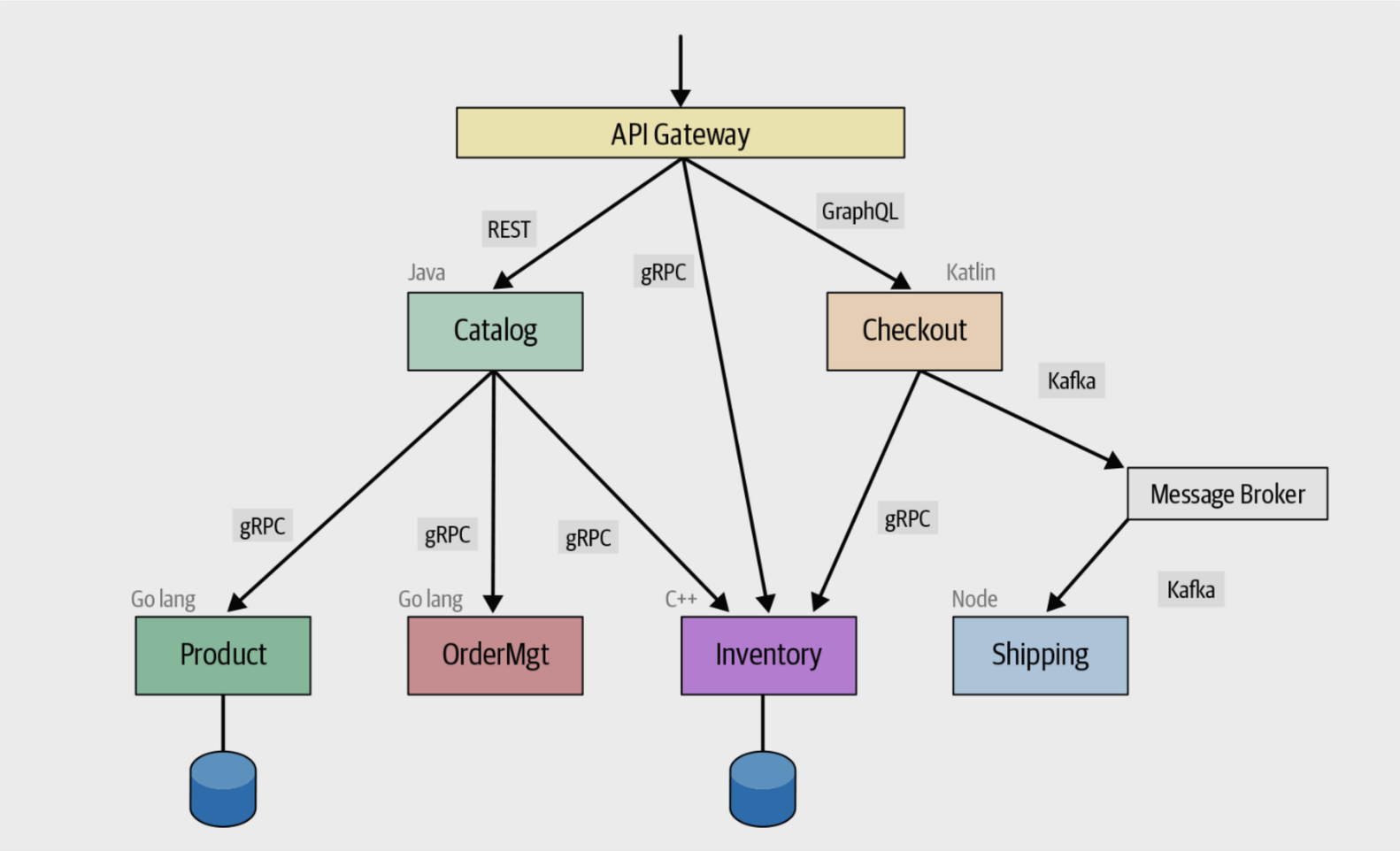
Caso gRPC


Proto Request
gRPC Stub
Ruby Client
Proto Response
gRPC Server
C++ Service
Proto Request
gRPC Stub
Proto Response(s)
Android-Java Client

Caso: gRPC


ProductInfo Service Definition
ProductInfo.proto
Consumer
Generate client stub
Generate Server Skeleton
Product Info
gRPC Stub
Protocol Buffer
over HTTP/2
gRPC Server
Java
Go

Caso gRPC


Client Machine
Server Machine
Client Process
Server Process
1. Client call to procedure
call getProduct("ABC")
getProduct(string ProductID)
call getProduct("ABC")
6. Stub makes local call to "getProduct" function
Encoded Message
10110...10
5. Stub unpacks the message
2. Stub builds the message
Request Details:
Service Name: Product Info
Function Name: get Product()
Value: "ABC"
Decoded Message
Message Headers
:method POST
:path /ProductInfo/getProduct
4. Server machine hands the message to server stub
HTTP/2 Connection
3. Message is sent across the network

Escenario Común de Microservicios